Команды пользователя

- Geant4 имеет встроенный набор команд, управляющих моделированием
  - уровень диагностики
  - изменение геометрических параметров модели
  - изменение списка учитываемых процессов
  -
- Этот набор может быть расширен пользователем
- Команды могут использоваться в интерактивном режиме, считываться из файла сценария или быть запрограммированы в коде

За обработку встроенных команд отвечает класс G4UImanager.

необходимо вызвать статический метод данного класса, возвращающий указатель на уже существующий объект данного класса `G4UImanager *UImanager = G4UImanager::GetUIpointer();`

<u>Вызов команд напрямую из кода</u>

UImanager->ApplyCommand(“units/list”);

Данным способом можно воспользоваться из любой части кода, однако надо учитывать состояние ядра Geant4 в данный момент.

во время цикла событий нельзя вызывать команды на изменение геометрии или используемых физических процессов.

Команды пользователя

Вызов команд из внешних файлов

В Geant4 реализована поддержка внешних файлов команд.

Такие файлы обычно имеют расширение *.mac.

Использование этих файлов удобно, например, при настройке сцены отрисовки.

Заранее можно выбрать средство отрисовки, задать необходимый угол обзора, добавить оси, логотипы всё что приходилось бы делать вручную из командной строки при каждом запуске

Использование файла (vis.mac) с командами в проекте:

UImanager->ApplyCommand("control/execute vis.mac");

Необходимо вызвать метод ApplyCommand() с командой control/execute, в качестве аргумента которой указать путь до файла.

Псевдонимы – это строковые переменные, которые можно использовать как аргументы команд.

/control/alias <aliasName> <aliasValue> где aliasName — имя переменной aliasValue — значение.

Путь, например, значение количества частиц в запуске передаётся сразу в несколько команд, как параметр. Тогда содержимое внешнего файла с командами может быть следующим (переменная псевдоним заключается в фигурные скобки):

```
/control/alias numOfPat 1000

/run/beamOn {numOfPat}
/my_com/com1 {numOfPat}
/my_com/com2 {numOfPat}
```

Команды пользователя

Использование цикла

/control/loop <macroFile> <counterName> <initialValue> <finalValue> <stepSize>

macroFile — представляет собой другой макро-файл, являющийся телом цикла

counterName — переменная псевдоним используемая в цикле

initialValue — стартовое значение псевдонима

finalValue — конечно значение псевдонима

stepSize- шаг для псевдонима

пусть нужно осуществить несколько запусков подряд, причем количество частиц в запуске должно увеличиваться на 100

Основной внешний файл (run1.mac) будет содержать команду

/control/loop run2.mac numOfPat 100 1000 100

где run2.mac представляет собой тело цикла и содержит команду:

/run/beamOn {numOfPat}

Команды пользователя

Данная команда устанавливает уровень подробности выводимой информации равным 1

командная директория может содержать внутренние каталоги, следовательно командой является всегда самый последний аргумент

Очистка экрана

- Параметры команд могут быть следующих типов: **G4String**, **G4int**, **G4bool** и **G4double**
- Пробел играет роль разделителя
- Двойные-кавычки (“”) используются для string-параметров, содержащих пробел

Параметр команды может быт не указан, тогда для пропущенного параметра используется значение по умолчанию

Информация о всех командах: /control/manual

Команды пользователя

Категории команд

/control	управление интерфейсом
/units	система единиц
/geometry	навигация и проверка перекрытий объемов
/tracking	управление объектами TrackingManager и SteppingManager
/event	управление объектом EventManager
/cuts	управление порогами рождения частиц
/run	управление сеансом
/random	управление генератором случайных чисел
/material	просмотр списка и свойств материалов
/particle	изменение списков и свойств частиц
/process	изменение списка активных процессов
/vis	визуализация
/gun	управление генератором первичной вершины
/hits	управление детектирующими объемами
/score	работа со счетчиками

Команды пользователя

/control

1) **execute** * Execute a macro file.
2) **loop** * Execute a macro file more than once.
3) **foreach** * Execute a macro file more than once.
4) **suppressAbortion** * Suppress the program abortion caused by G4Exception.
5) **verbose** * Applied command will also be shown on screen.
6) **saveHistory** * Store command history to a file.
7) **stopSavingHistory** * Stop saving history file.
8) **alias** * Set an alias.
9) **unalias** * Remove an alias.
10) **listAlias** * List aliases.
11) **shell** * Execute a (Unix) SHELL command.
12) **manual** * Display all of sub-directories and commands.
13) **createHTML** * Generate HTML files for all of sub-directories and commands.
14) **maximumStoredHistory** * Set maximum number of stored UI commands.

Выполнение сценария:

/control/execute имя_файла_сценария

/control/execute vis.mac

Выполнение команды оболочки Unix

/control/shell команда

/control/shell ls -la

Команды пользователя

Вывод списка используемых единиц:

/units/list

----- The Table of Units -----

category: Length
parsec (pc) = 3.08568e+19
kilometer (km) = 1e+06
meter (m) = 1000
centimeter (cm) = 10
millimeter (mm) = 1
micrometer (mum) = 0.001
nanometer (nm) = 1e-06
angstrom (Ang) = 1e-07
fermi (fm) = 1e-12

category: Surface
kilometer2 (km2) = 1e+12
meter2 (m2) = 1e+06

. . .

/geometry/test - проверка геометрии на перекрытие объемов

/tracking/abort - прервать моделирование текущего трека

/tracking/resume - возобновить моделирование текущего трека

/tracking/storeTrajectory – сохранять траектории

/tracking/verbose – уровень диагностики трекинга частиц

/tracking/verbose уровень_детализации

Уровни детализации диагностики

0 : Сообщения отсутствуют
1 : Минимальная информация о каждом шаге
2 : Уровень=1 + информация о вторичных частицах
3 : Дополнительно информация о состоянии в начале и конце каждого шага
4 : Дополнительно информация о состоянии в начале и конце каждого шага с детализацией по отдельным процессам
5 : Дополнительно информация об оценках длины шага согласно каждому отдельному процессу

/tracking/storeTrajectory 0 - траектории частиц не сохраняются

/tracking/storeTrajectory 1 – траектории частиц сохраняются и могут быть визуализированы

Команды пользователя

/event/stack/status – вывести подробную информацию о стеках треков в событии

/event/stack/clear - сбросить стеки

/event/abort - прекратить моделирование текущего события

/event/verbose 0 - нет сообщений диагностики

/event/verbose 1 - вывод информации о стеках

/material/nist/printElement аргумент – символ элемента либо all

/material/nist/printElementZ аргумент - Z

/material/nist/listMaterials аргумент- simple, compound, hep,all

/material/g4/printElement

/material/g4/printMaterial

/run

Команды пользователя

Подкаталог :

1) **/run/particle/** управление списком физических процессов

Команды :

2) **initialize** * Инициализация ядра Geant4
3) **beamOn** * Начало сеанса
4) **verbose** * Уровень диагностики объекта G4RunManager.
5) **dumpRegion** * информация об объектах G4Region
6) **dumpCouples** * вывод таблицы соответствия “материал-порог”
7) **optimizeGeometry** * флаг оптимизации геометрии
8) **breakAtBeginOfEvent** * пауза в начале каждого события
9) **breakAtEndOfEvent** * пауза в конце каждого события
10) **abort** * прервать текущий сеанс
11) **abortCurrentEvent** * прервать текущее событие
12) **geometryModified** * заново создать модель детектора
13) **physicsModified** * пересчитать таблицы физических процессов
14) **cutoffModified** * - не применяется
15) **randomNumberStatusDirectory** * каталог для сохранения файла с начальным значением генератора случайных чисел
16) **storeRandomNumberStatus** * установить сохранение начального значения генератора в начале сеанса и в начале события
17) **restoreRandomNumberStatus** * считать начальное значение генератора из файла
18) **setCut** * установить значение порога по умолчанию
19) **setCutForRegion** * установить значение порога для объекта G4Region

Основная команда при моделировании

Начать моделирование 1000 событий

/run/beamOn 1000

Начать моделирование 1000 событий, и после каждого из первых 100 событий выполнить сценарий event.mac

/run/beamOn 1000 event.mac 100

Команды пользователя (Настройка визуализации)

```
/vis/open OGL 600x600-0+0
#/vis/open DAWNFILE
#/vis/open HepRepFile
#/vis/open VRML2FILE
/vis/viewer/set/autoRefresh false
/vis/verbose errors
/vis/drawVolume
#/vis/viewer/set/viewpointThetaPhi 90. 0.
#/vis/viewer/zoom 2.
#/vis/viewer/set/style wireframe
#/vis/scene/add/axes 0 0 0 1 m
/vis/scene/add/trajectories smooth
/vis/modeling/trajectories/create/drawByCharge
/vis/modeling/trajectories/drawByCharge-0/default/setDrawStepPts true
/vis/modeling/trajectories/drawByCharge-0/default/setStepPtsSize 2
#/vis/scene/add/hits
#/vis/filtering/trajectories/create/particleFilter
#/vis/filtering/trajectories/particleFilter-0/add gamma
#/vis/filtering/trajectories/particleFilter-0/invert true
#/vis/modeling/trajectories/create/drawByParticleID
#/vis/modeling/trajectories/drawByParticleID-0/set e- blue
#/vis/scene/endOfEventAction accumulate
/vis/viewer/set/autoRefresh true
/vis/verbose warnings
#/vis/viewer/flush
```

Команды пользователя (Настройка визуализации)

```
/vis/open OGL 600x600-0+0
#/vis/open DAWNFILE
#/vis/open HepRepFile
#/vis/open VRML2FILE
/vis/viewer/set/autoRefresh false
/vis/verbose errors
/vis/drawVolume
#/vis/viewer/set/viewpointThetaPhi 90. 0.
#/vis/viewer/zoom 2.
#/vis/viewer/set/style wireframe
#/vis/scene/add/axes 0 0 0 1 m
/vis/scene/add/trajectories smooth
/vis/modeling/trajectories/create/drawByCharge
/vis/modeling/trajectories/drawByCharge-0/default/setDrawStepPts true
/vis/modeling/trajectories/drawByCharge-0/default/setStepPtsSize 2
#/vis/scene/add/hits
#/vis/filtering/trajectories/create/particleFilter
#/vis/filtering/trajectories/particleFilter-0/add gamma
#/vis/filtering/trajectories/particleFilter-0/invert true
#/vis/modeling/trajectories/create/drawByParticleID
#/vis/modeling/trajectories/drawByParticleID-0/set e- blue
#/vis/scene/endOfEventAction accumulate
/vis/viewer/set/autoRefresh true
/vis/verbose warnings
#/vis/viewer/flush
```

Simplest Command Sequence

Команды пользователя (Настройка визуализации)

To Open Visualization

- To Open a Driver

```
/vis/open <driver name>
```

- for example

```
/vis/open OGL
/vis/open DAWNFILE
/vis/open HepRepFile
/vis/open VRML2FILE
```

- The set of available drivers is listed when you first start Geant4,
  but you can also get this list with the command:

```
help /vis/open
```

Команды пользователя (Настройка визуализации)

More about Open, and about DrawVolume

- Some drivers have additional options at open

  e.g., for OpenGL, can specify size and location of window

  ```
  /vis/open OGL 600x600-0+0
  ```

- You can open more than one viewer at a time:

  ```
  /vis/open OGL
  /vis/open HepRepFile
  ```

  - To see what viewers you then have:

    ```
    /vis/viewer/list
    ```

  - To select which viewer is the current one:

    ```
    /vis/viewer/select viewer-0
    /vis/viewer/select viewer-1
    ```

  - All other vis commands affect only the currently selected viewer

- To draw the entire detector geometry:

  ```
  /vis/drawVolume
  ```

Команды пользователя (Настройка визуализации)

```
/vis/open OGL 600x600-0+0
#/vis/open DAWNFILE
#/vis/open HepRepFile
#/vis/open VRML2FILE
/vis/viewer/set/autoRefresh false
/vis/verbose errors
/vis/drawVolume
/vis/viewer/set/viewpointThetaPhi 90. 0.
/vis/viewer/zoom 2.
/vis/viewer/set/style wireframe
#/vis/scene/add/axes 0 0 0 1 m
/vis/scene/add/trajectories smooth
/vis/modeling/trajectories/create/drawByCharge
/vis/modeling/trajectories/drawByCharge-0/default/setDrawStepPts true
/vis/modeling/trajectories/drawByCharge-0/default/setStepPtsSize 2
#/vis/scene/add/hits
#/vis/filtering/trajectories/create/particleFilter
#/vis/filtering/trajectories/particleFilter-0/add gamma
#/vis/filtering/trajectories/particleFilter-0/invert true
#/vis/modeling/trajectories/create/drawByParticleID
#/vis/modeling/trajectories/drawByParticleID-0/set e- blue
#/vis/scene/endOfEventAction accumulate
/vis/viewer/set/autoRefresh true
/vis/verbose warnings
#/vis/viewer/flush
```

Controlling the viewpoint and zoom

Команды пользователя (Настройка визуализации)

The /vis/viewer/… Commands

- Set view angles

```
/vis/viewer/set/viewpointThetaPhi <theta_angle>
<phi_angle>
```

- Zoom

```
/vis/viewer/zoom <scale factor>
```

- Reset viewpoint

```
/vis/viewer/reset
```

- Set drawing style

```
/vis/viewer/set/style <style>
```

  - Options for style: wireframe , surface
  - but note that this will not affect volumes that have style explicitly forced by “setForceWireframe” or “setForceSolid” commands in the c++ code

Методы объекта G4VisAttributes

Команды пользователя (Настройка визуализации)

```
/vis/open OGL 600x600-0+0
#/vis/open DAWNFILE
#/vis/open HepRepFile
#/vis/open VRML2FILE
/vis/viewer/set/autoRefresh false
/vis/verbose errors
/vis/drawVolume
/vis/viewer/set/viewpointThetaPhi 90. 0.
/vis/viewer/zoom 2.
/vis/viewer/set/style wireframe
/vis/scene/add/axes 0 0 0 1 m
/vis/scene/add/trajectories smooth
/vis/modeling/trajectories/create/drawByCharge
/vis/modeling/trajectories/drawByCharge-0/default/setDrawStepPts true
/vis/modeling/trajectories/drawByCharge-0/default/setStepPtsSize 2
/vis/scene/add/hits
#/vis/filtering/trajectories/create/particleFilter
#/vis/filtering/trajectories/particleFilter-0/add gamma
#/vis/filtering/trajectories/particleFilter-0/invert true
#/vis/modeling/trajectories/create/drawByParticleID
#/vis/modeling/trajectories/drawByParticleID-0/set e- blue
#/vis/scene/endOfEventAction accumulate
/vis/viewer/set/autoRefresh true
/vis/verbose warnings
#/vis/viewer/flush
```

Add axes, trajectories and hits

Команды пользователя (Настройка визуализации)

Axes, Trajectories and Hits

- Axes

  ```
  /vis/scene/add/axes <x_origin> <y_origin>
  <z_origin> <size> <units>
  ```

- Trajectories

  ```
  /vis/scene/add/trajectories
  ```

  - By default, trajectories are redrawn at every event, try: `/run/beamOn 1`

- Hits (if application has hits defined)

  ```
  /vis/scene/add/hits
  ```

Команды пользователя (Настройка визуализации)

```
/vis/open OGL 600x600-0+0
#/vis/open DAWNFILE
#/vis/open HepRepFile
#/vis/open VRML2FILE
/vis/viewer/set/autoRefresh false
/vis/verbose errors
/vis/drawVolume
/vis/viewer/set/viewpointThetaPhi 90. 0.
/vis/viewer/zoom 2.
/vis/viewer/set/style wireframe
/vis/scene/add/axes 0 0 0 1 m
/vis/scene/add/trajectories smooth
/vis/modeling/trajectories/create/drawByCharge
/vis/modeling/trajectories/drawByCharge-0/default/setDrawStepPts true
/vis/modeling/trajectories/drawByCharge-0/default/setStepPtsSize 2
/vis/scene/add/hits
#/vis/filtering/trajectories/create/particleFilter
#/vis/filtering/trajectories/particleFilter-0/add gamma
#/vis/filtering/trajectories/particleFilter-0/invert true
/vis/modeling/trajectories/create/drawByParticleID
/vis/modeling/trajectories/drawByParticleID-0/set e- blue
/vis/scene/endOfEventAction accumulate
/vis/viewer/set/autoRefresh true
/vis/verbose warnings
#/vis/viewer/flush
```

Visualizing step points

Команды пользователя (Настройка визуализации)

Visualizing Step Points

- By default, the trajectory is drawn just as a line
- To also show the step points:

```
/vis/modeling/trajectories/create/drawByCharge
/vis/modeling/trajectories/drawByCharge-0/default/
setDrawStepPts true
/vis/modeling/trajectories/drawByCharge-0/default/
setStepPtsSize 2
```

  This syntax is complicated because it actually supports many more options on how trajectories and step points should be modeled.

- Trajectories and step points can contain additional, non-displayed information
  - such as particle id, momentum, etc.
  - shown when you pick on the trajectory in some visualization drivers.
- Turn on extra info with parameter rich:

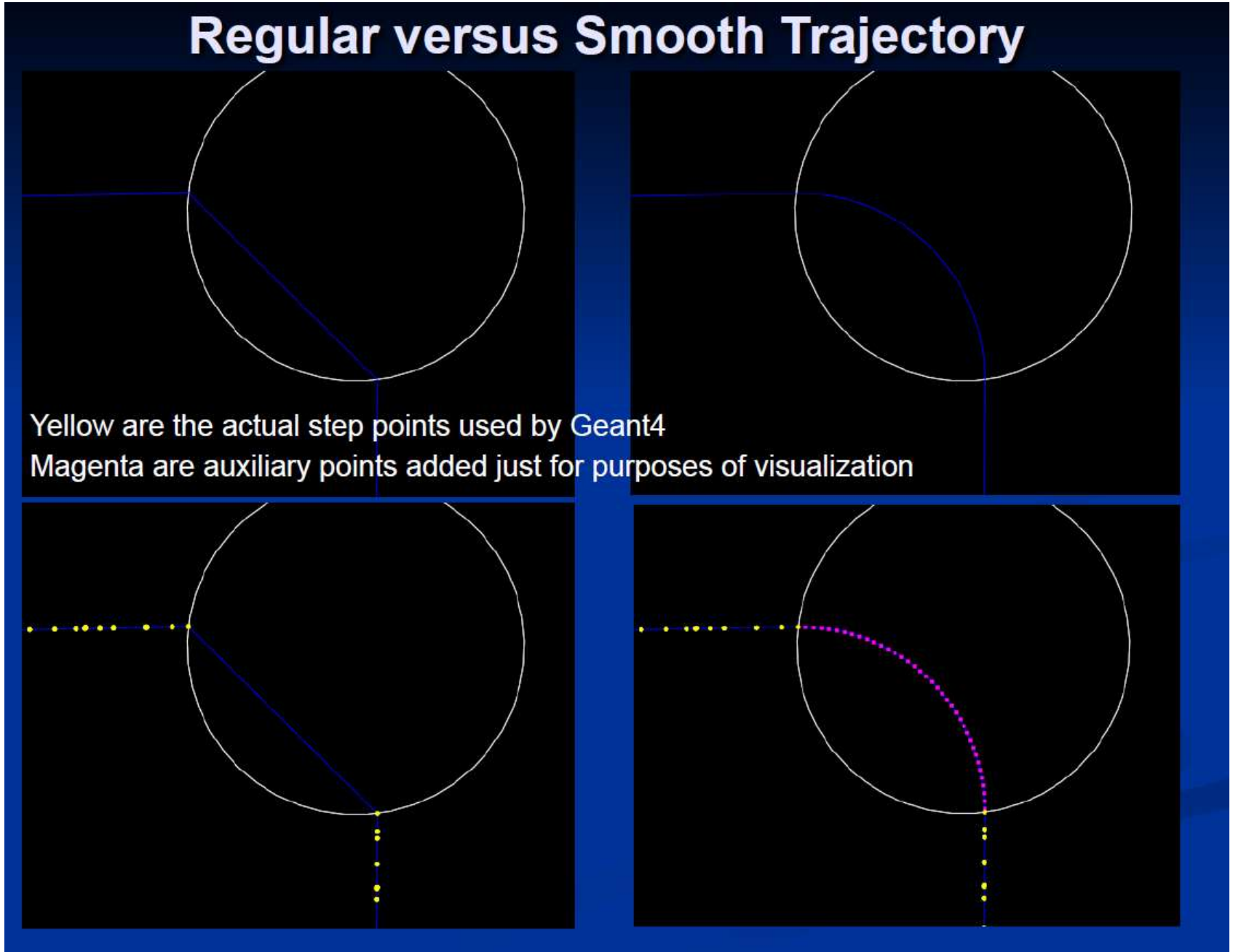
```
/vis/scene/add/trajectories rich
```

Команды пользователя (Настройка визуализации)

```
/vis/open OGL 600x600-0+0
#/vis/open DAWNFILE
#/vis/open HepRepFile
#/vis/open VRML2FILE
/vis/viewer/set/autoRefresh false
/vis/verbose errors
/vis/drawVolume
/vis/viewer/set/viewpointThetaPhi 90. 0.
/vis/viewer/zoom 2.
/vis/viewer/set/style wireframe
/vis/scene/add/axes 0 0 0 1 m
/vis/scene/add/trajectories smooth
/vis/modeling/trajectories/create/drawByCharge
/vis/modeling/trajectories/drawByCharge-0/default/setDrawStepPts true
/vis/modeling/trajectories/drawByCharge-0/default/setStepPtsSize 2
/vis/scene/add/hits
#/vis/filtering/trajectories/create/particleFilter
#/vis/filtering/trajectories/particleFilter-0/add gamma
#/vis/filtering/trajectories/particleFilter-0/invert true
/vis/modeling/trajectories/create/drawByParticleID
/vis/modeling/trajectories/drawByParticleID-0/set e- blue
/vis/scene/endOfEventAction accumulate
/vis/viewer/set/autoRefresh true
/vis/verbose warnings
```

Smooth Trajectories

Команды пользователя (Настройка визуализации)



Команды пользователя (Настройка визуализации)

Smooth Trajectories

- By default, the trajectory is represented as a series of line segments from one step point to the next.
  - For the case of strong fields, this may result in jagged looking tracks.
- Can ask visualization to smooth the lines with:

```
/vis/scene/add/trajectories smooth
```

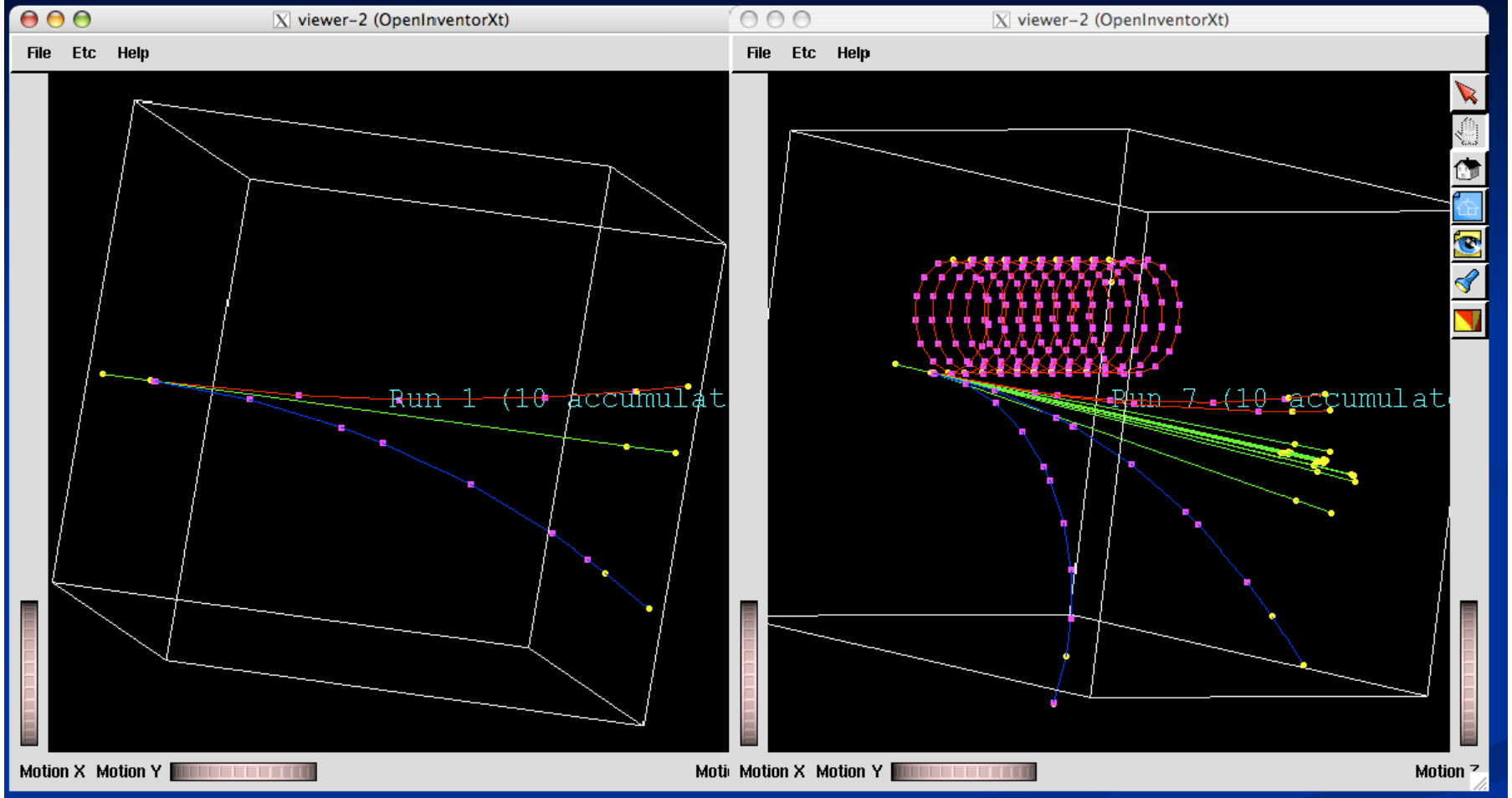
The extra points are not actual Geant4 step points.
Smooth does not change how Geant4 actually does its stepping.
These extra “auxiliary points” are only added to make a smoother line.

- Trajectories can be smooth, rich or both:

```
/vis/scene/add/trajectories smooth rich
```

Команды пользователя (Настройка визуализации)

Smooth Trajectory Makes Big Difference for Trajectories that Loop in a Magnetic Field



- Yellow dots are the actual step points used by Geant4
- Magenta dots are auxiliary points added just for purposes of visualization

Команды пользователя (Настройка визуализации)

```
/vis/open OGL 600x600-0+0
#/vis/open DAWNFILE
#/vis/open HepRepFile
#/vis/open VRML2FILE
/vis/viewer/set/autoRefresh false
/vis/verbose errors
/vis/drawVolume
/vis/viewer/set/viewpointThetaPhi 90. 0.
/vis/viewer/zoom 2.
/vis/viewer/set/style wireframe
/vis/scene/add/axes 0 0 0 1 m
/vis/scene/add/trajectories smooth
/vis/modeling/trajectories/create/drawByCharge
/vis/modeling/trajectories/drawByCharge-0/default/setDrawStepPts true
/vis/modeling/trajectories/drawByCharge-0/default/setStepPtsSize 2
/vis/scene/add/hits
#/vis/filtering/trajectories/create/particleFilter
#/vis/filtering/trajectories/particleFilter-0/add gamma
#/vis/filtering/trajectories/particleFilter-0/invert true
/vis/modeling/trajectories/create/drawByParticleID
/vis/modeling/trajectories/drawByParticleID-0/set e- blue
#/vis/scene/endOfEventAction accumulate
/vis/viewer/set/autoRefresh true
/vis/verbose warnings
#/vis/viewer/flush
```

Basic trajectory modeling

Команды пользователя (Настройка визуализации)

Basic Trajectory Modeling

- By default, trajectories are color-coded by charge
  - /vis/modeling/trajectories/drawByParticleID-0/set e- red
  - /vis/modeling/trajectories/drawByParticleID-0/set e+ blue
  - /vis/modeling/trajectories/drawByParticleID-0/set proton cyan
  - /vis/modeling/trajectories/drawByParticleID-0/set gamma green
  - /vis/modeling/trajectories/drawByParticleID-0/set neutron yellow
  - /vis/modeling/trajectories/drawByParticleID-0/set pi+ magenta
  - /vis/modeling/trajectories/drawByParticleID-0/set pi- magenta
  - /vis/modeling/trajectories/drawByParticleID-0/set pi0 magenta
  - # and everything else still grey
- But you can choose other modeling options, such as color by particle ID
  - /vis/modeling/trajectories/create/drawByParticleID
  - /vis/modeling/trajectories/drawByParticleID-0/set e- blue

Команды пользователя (Настройка визуализации)

```
/vis/open OGL 600x600-0+0
#/vis/open DAWNFILE
#/vis/open HepRepFile
#/vis/open VRML2FILE
/vis/viewer/set/autoRefresh false
/vis/verbose errors
/vis/drawVolume
/vis/viewer/set/viewpointThetaPhi 90. 0.
/vis/viewer/zoom 2.
/vis/viewer/set/style wireframe
/vis/scene/add/axes 0 0 0 1 m
/vis/scene/add/trajectories smooth
/vis/modeling/trajectories/create/drawByCharge
/vis/modeling/trajectories/drawByCharge-0/default/setDrawStepPts true
/vis/modeling/trajectories/drawByCharge-0/default/setStepPtsSize 2
/vis/scene/add/hits
#/vis/filtering/trajectories/create/particleFilter
#/vis/filtering/trajectories/particleFilter-0/add gamma
#/vis/filtering/trajectories/particleFilter-0/invert true
/vis/modeling/trajectories/create/drawByParticleID
/vis/modeling/trajectories/drawByParticleID-0/set e- blue
/vis/scene/endOfEventAction accumulate
/vis/viewer/set/autoRefresh true
/vis/verbose warnings
#/vis/viewer/flush
```

Accumulating trajectories and hits

Команды пользователя (Настройка визуализации)

Accumulating Trajectories and Hits

- By default, you will get a drawing after each event. To instead get just one drawing with all of the accumulated events from that run

```
/vis/scene/endOfEventAction accumulate
```

- To even suppress that one drawing from the end of the /run/beamOn, use

```
/vis/scene/endOfRunAction accumulate
```

- When you actually want to draw, you then have to explicitly issue the command

```
/vis/viewer/flush
```

Команды пользователя (Настройка визуализации)

```
/vis/open OGL 600x600-0+0
#/vis/open DAWNFILE
#/vis/open HepRepFile
#/vis/open VRML2FILE
/vis/viewer/set/autoRefresh false
/vis/verbose errors
/vis/drawVolume
/vis/viewer/set/viewpointThetaPhi 90. 0.
/vis/viewer/zoom 2.
/vis/viewer/set/style wireframe
/vis/scene/add/axes 0 0 0 1 m
/vis/scene/add/trajectories smooth
/vis/modeling/trajectories/create/drawByCharge
/vis/modeling/trajectories/drawByCharge-0/default/setDrawStepPts true
/vis/modeling/trajectories/drawByCharge-0/default/setStepPtsSize 2
/vis/scene/add/hits
/vis/filtering/trajectories/create/particleFilter
/vis/filtering/trajectories/particleFilter-0/add gamma
/vis/filtering/trajectories/particleFilter-0/invert true
/vis/modeling/trajectories/create/drawByParticleID
/vis/modeling/trajectories/drawByParticleID-0/set e- blue
/vis/scene/endOfEventAction accumulate
/vis/viewer/set/autoRefresh true
/vis/verbose warnings
#/vis/viewer/flush
```

Filtering Trajectories

Команды пользователя (Настройка визуализации)

Filtering Trajectories

- By default, all trajectories are drawn
- You apply a filter so that only certain trajectories are drawn:

```
/vis/filtering/trajectories/create/particleFilter
/vis/filtering/trajectories/particleFilter-0/add gamma
```

- The above adds a filter that only allows gammas to draw
- To instead do the opposite, drawing everything except gammas,
  include the above, but also add the following:

```
/vis/filtering/trajectories/particleFilter-0/invert
true
```

Команды пользователя (Настройка визуализации)

```
/vis/open OGL 600x600-0+0
#/vis/open DAWNFILE
#/vis/open HepRepFile
#/vis/open VRML2FILE
/vis/viewer/set/autoRefresh false
/vis/verbose errors
/vis/drawVolume
/vis/viewer/set/viewpointThetaPhi 90. 0.
/vis/viewer/zoom 2.
/vis/viewer/set/style wireframe
/vis/scene/add/axes 0 0 0 1 m
/vis/scene/add/trajectories smooth
/vis/modeling/trajectories/create/drawByCharge
/vis/modeling/trajectories/drawByCharge-0/default/setDrawStepPts true
/vis/modeling/trajectories/drawByCharge-0/default/setStepPtsSize 2
/vis/scene/add/hits
/vis/filtering/trajectories/create/particleFilter
/vis/filtering/trajectories/particleFilter-0/add gamma
/vis/filtering/trajectories/particleFilter-0/invert true
/vis/modeling/trajectories/create/drawByParticleID
/vis/modeling/trajectories/drawByParticleID-0/set e- blue
/vis/scene/endOfEventAction accumulate
/vis/viewer/set/autoRefresh true
/vis/verbose warnings
/vis/viewer/flush
```

To force output of a new file

Команды пользователя (Настройка визуализации)

To Force Output of a New File

- There are two classes of Geant4 visualization drivers:
  - Immediate drivers: draw directly to the screen (such as OpenGL)
  - File-based drivers: create a file on disk (HepRepFile, VRML2FILE)
- For immediate drivers you see the results or your /vis commands immediately
- For the file-based drivers:
  - the default is to only create a new file (showing your changes) when you do `/run/beamOn`
- If you want to see visualization at some other time, such as after you set up geometry, but before you do /run/beamOn, use: `/vis/viewer/flush`

Команды пользователя (Настройка визуализации)

```
/vis/open OGL 600x600-0+0
#/vis/open DAWNFILE
#/vis/open HepRepFile
#/vis/open VRML2FILE
/vis/viewer/set/autoRefresh false
/vis/verbose errors
/vis/drawVolume
/vis/viewer/set/viewpointThetaPhi 90. 0.
/vis/viewer/zoom 2.
/vis/viewer/set/style wireframe
/vis/scene/add/axes 0 0 0 1 m
/vis/scene/add/trajectories smooth
/vis/modeling/trajectories/create/drawByCharge
/vis/modeling/trajectories/drawByCharge-0/default/setDrawStepPts true
/vis/modeling/trajectories/drawByCharge-0/default/setStepPtsSize 2
/vis/scene/add/hits
/vis/filtering/trajectories/create/particleFilter
/vis/filtering/trajectories/particleFilter-0/add gamma
/vis/filtering/trajectories/particleFilter-0/invert true
/vis/modeling/trajectories/create/drawByParticleID
/vis/modeling/trajectories/drawByParticleID-0/set e- blue
/vis/scene/endOfEventAction accumulate
/vis/viewer/set/autoRefresh true
/vis/verbose warnings
/vis/viewer/flush
```

To avoid excessive redrawing on immediate viewers

Команды пользователя (Настройка визуализации)

To Avoid Excessive Redrawing

- By default, immediate viewers will redraw after every vis command that might change the view, such as:

```
/vis/drawVolume
/vis/viewer/set/viewpointThetaPhi 90. 0.
/vis/viewer/zoom 2.
/vis/viewer/set/style wireframe
/vis/scene/add/axes 0 0 0 1 m
```

- If the geometry is very complex (such as in some imported patient geometries for medical applications), this can result in slow performance
- To temporarily turn off this redrawing:

```
/vis/viewer/set/autoRefresh false
```

- And then once everything is set up:

```
/vis/viewer/set/autoRefresh true
```

Команды пользователя (Настройка визуализации)

```
/vis/open OGL 600x600-0+0
#/vis/open DAWNFILE
#/vis/open HepRepFile
#/vis/open VRML2FILE
/vis/viewer/set/autoRefresh false
/vis/verbose errors
/vis/drawVolume
/vis/viewer/set/viewpointThetaPhi 90. 0.
/vis/viewer/zoom 2.
/vis/viewer/set/style wireframe
/vis/scene/add/axes 0 0 0 1 m
/vis/scene/add/trajectories smooth
/vis/modeling/trajectories/create/drawByCharge
/vis/modeling/trajectories/drawByCharge-0/default/setDrawStepPts true
/vis/modeling/trajectories/drawByCharge-0/default/setStepPtsSize 2
/vis/scene/add/hits
/vis/filtering/trajectories/create/particleFilter
/vis/filtering/trajectories/particleFilter-0/add gamma
/vis/filtering/trajectories/particleFilter-0/invert true
/vis/modeling/trajectories/create/drawByParticleID
/vis/modeling/trajectories/drawByParticleID-0/set e- blue
/vis/scene/endOfEventAction accumulate
/vis/viewer/set/autoRefresh true
/vis/verbose warnings
/vis/viewer/flush
```

To turn off unwanted visualization messages on the console

Команды пользователя (Настройка визуализации)

To Turn off Unwanted Visualization Messages

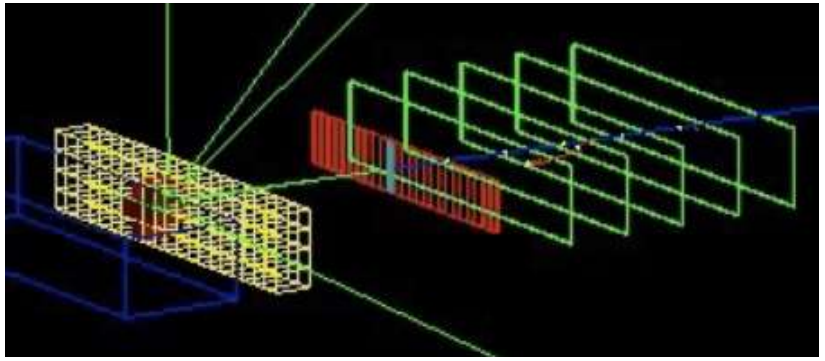
- You can control how many messages visualization puts on the console by:

```
/vis/verbose <level>
0) quiet,        // Nothing is printed.
1) startup,      // Startup and endup messages are printed...
2) errors,       // ...and errors...
3) warnings,     // ...and warnings...
4) confirmations, // ...and confirming messages...
5) parameters,   // ...and parameters of scenes and views...
6) all           // ...and everything available.
```

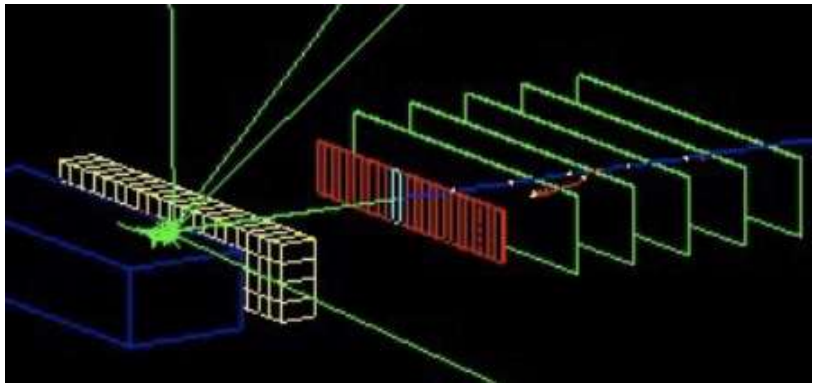
Команды пользователя (Настройка визуализации)

Драйвер OpenGL обеспечивает возможность удаления внутренних линий геометрии и трека

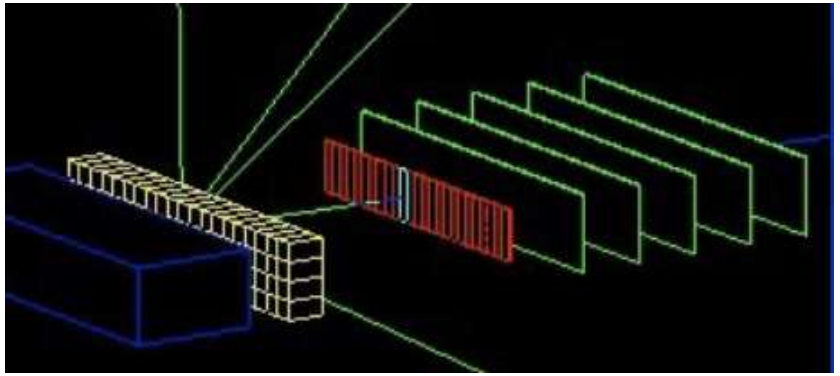
По умолчанию удаление внутренних линий геометрии и трека отключено



Удаление внутренних линий геометрии
/vis/viewer/set/hiddenEdge 1



Удаление внутренних линий трека
/vis/viewer/set/hiddenMarker 1



Создание новых команд

В Geant4 предусмотрена возможность по созданию новых команд, в добавление к уже существующим. Для этого следует создать объект класса, унаследованного от **G4UImessenger**, а для реализации действий, предусмотренных данными командами, следует переопределить метод **SetNewValue(G4UIcommand*, G4String)**.

<u>Пример</u> взаимодействий новой команды с классом G4VUserPrimaryGenerationAction, где задаются параметры запуска частицы.

Чтобы реализовать взаимодействие следует обеспечить связь между потомками классов G4VUserPrimaryGenerationAction и G4UImessenger.

Указатель primaryPart содержит адрес класса-потомка PrimaryPart класса G4VUserPrimaryGenerationAction

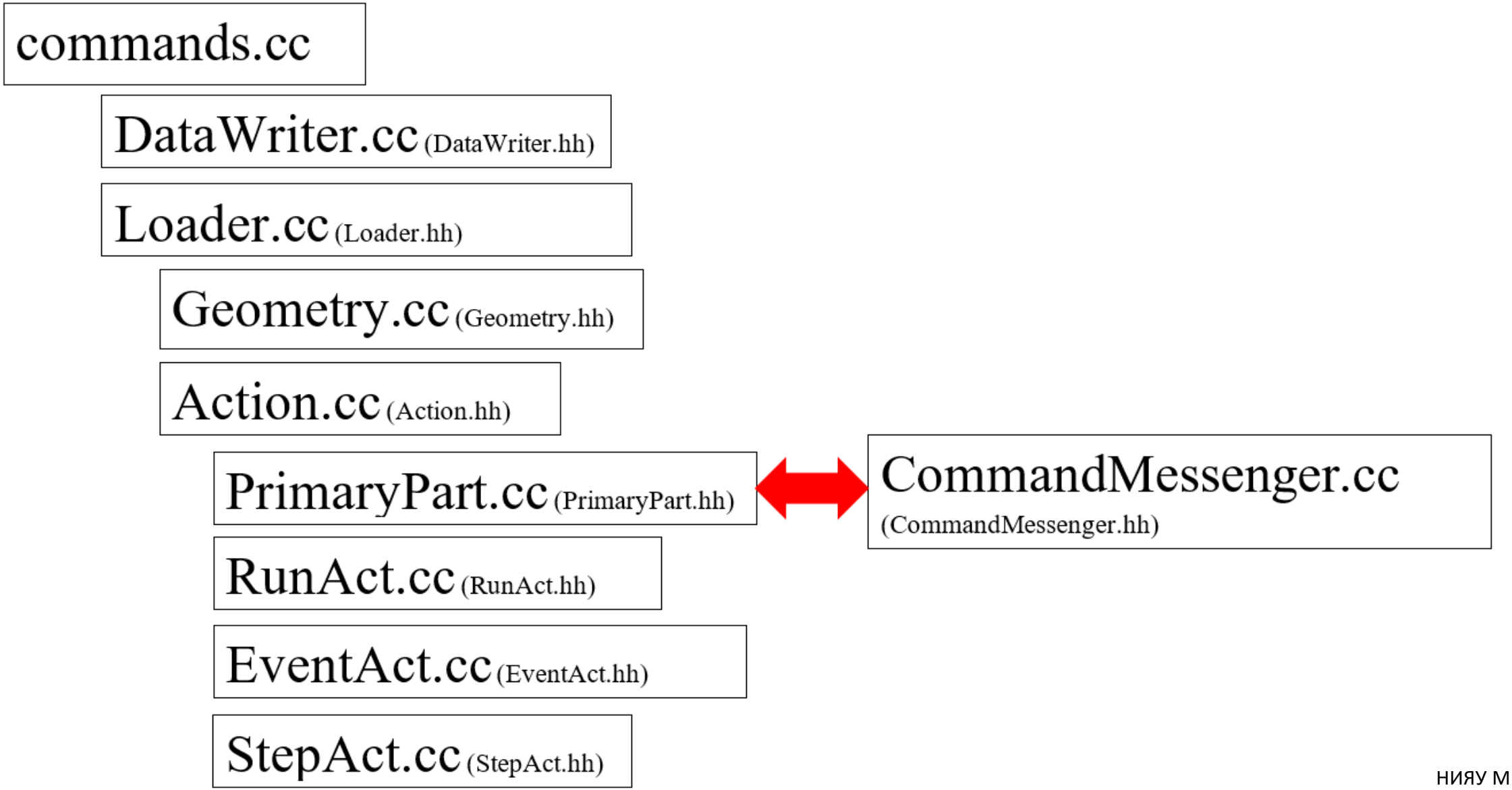
CommandMessenger.hh

```
class CommandMessenger: public G4UImessenger
{
 public:
  std::ofstream *f_messenger;
  explicit CommandMessenger(PrimaryPart*);
  ~CommandMessenger();

  void SetNewValue(G4UIcommand*, G4String);
  G4String GetCurrentValue(G4UIcommand*);
  private:
   PrimaryPart* primaryPart;
   G4UIdirectory* primaryPartCommand;
   G4UIcmdWithADoubleAndUnit* my_cmd;
};
```

Создание новых команд

Проект commands



Создание новых команд

Реализация новой команды начинается с создания директории с использованием класса G4UIdirectory.

Для созданной директории команды можно добавить описание с использованием метода SetGuidance(const* char)

Все классы команд в Geant4 наследуют класс G4UIcommand.

Для всех основных типов данных параметров создаваемой команды используются потомки этого класса:

- G4UIcmdWithoutParameter
- G4UIcmdWithABool
- G4UIcmdWithAInteger
- G4UIcmdWithAString
- G4UIcmdWith3Vector
- G4UIcmdWith3VectorAndUnit
- G4UIcmdWithADouble
- G4UIcmdWithADoubleAndUnit

CommandMessenger.hh

```
class CommandMessenger: public G4UImessenger
{
 public:
  std::ofstream *f_messenger;
  explicit CommandMessenger(PrimaryPart*);
  ~CommandMessenger();

  void SetNewValue(G4UIcommand*, G4String);
  G4String GetCurrentValue(G4UIcommand*);
  private:
   PrimaryPart* primaryPart;
   G4UIdirectory* primaryPartCommand;
   G4UIcmdWithADoubleAndUnit* my_cmd;
};
```

Создание новых команд

CommandMessenger.cc

Конструктор класса CommandMessenger

```
CommandMessenger::CommandMessenger(PrimaryPart* pP):primaryPart(pP)
{
 this->f_messenger=primaryPart->f_prim;
 (*f_messenger) << "Hi from messenger!" << G4endl;
  primaryPartCommand = new G4UIdirectory("/PrimaryPart/");
  primaryPartCommand->SetGuidance("User Primary Particles commands");

  my_cmd = new G4UIcmdWithADoubleAndUnit("/PrimaryPart/my_cmd",this);
  my_cmd->SetGuidance("It's my command!");
  my_cmd->SetParameterName("Energy",true,true);
  my_cmd->SetDefaultValue(1.);
  my_cmd->SetDefaultUnit("keV");
  my_cmd->SetUnitCandidates("ev keV MeV GeV TeV");
  my_cmd->SetRange("Energy != 0.");
}
```

- значение параметра по умолчанию;
- размерность параметра по умолчанию;
- список используемых размерностей параметра;
- диапазон допустимых значений параметра, описанного в SetParmeterName

Инициализация потока вывода класса CommandMessenger через поток вывода класса PrimaryPart

Создание директории и описания

Инициализация команды

Указатель на существующий объект G4UImessenger

Полный путь/имя команды

Для команд доступно несколько методов для настройки справочной информации:

- описание команды отображаемое в режиме визуализации;
- имя параметра команды, параметр может не указываться, текущее значение используется по умолчанию

Создание новых команд

Чтобы команда изменила начальную энергию запуска частицы, нужно реализовать в классе Commandmessenger метод SetNewValue() и передавать в него значение из команды в случае её вызова.

Вывод информации в файл в случае вызова созданной команды my_cmd

Каждый из потомков класса G4UIcommand имеет метод для конвертации своего типа данных из G4String

CommandMessenger.cc

```
void CommandMessenger::SetNewValue(G4UIcommand* command,G4String newValue)
{
 if (command == my_cmd)
 {
  (*f_messenger) << "command value before " << (this->GetCurrentValue(my_cmd)) << G4endl;
  this->primaryPart->GetParticleGun()->SetParticleEnergy(my_cmd->GetNewDoubleValue(newValue));
  (*f_messenger) << "command value after " << (this->GetCurrentValue(my_cmd)) << G4endl;
 }
}
G4String CommandMessenger::GetCurrentValue(G4UIcommand* command)
{G4String cv;
 if (command==my_cmd)
 {cv=my_cmd->ConvertToString(this->primaryPart->GetParticleGun()->GetParticleEnergy(),"GeV");}
 return cv;
}
```

Создание новых команд

Для объекта класса PrimaryPart нужно ввести дополнительное «поле-указатель» на объект CommandMessenger

PrimaryPart.hh

```
class PrimaryPart: public G4VUserPrimaryGeneratorAction
{
 private:
 ……————————————
 CommandMessenger* commandMessenger;
 ……————————————
}
```

Инициализировать введённый объект в конструкторе класса PrimaryPart

PrimaryPart.cc

```
 PrimaryPart::PrimaryPart(std::ofstream& ofsa)
 {
  ……————————————————————————
  commandMessenger= CommandMessenger(this);
  ……————————————————————————
 }
```

Создание новых команд

Пример взаимодействий новой команды с классом G4VUserDetectorConstruction, где задаются параметры детекторов.

Заголовочный файл класса новой команды

```
#include <G4UImessenger.hh>
#include "Geometry.hh"

class Geometry;

class GeometryMessenger: public G4UImessenger {
public:
    GeometryMessenger(Geometry *det);

    ~GeometryMessenger();

    void SetNewValue(G4UIcommand *command, G4String newValue);
};
```

Инициализация указателя на объект класса GeometryMessenger в конструкторе класса Geometry

```
geometryMessenger = new GeometryMessenger(this);
```

Создание новых команд

Метод Construct в классе описания геометрии

в первых четырех строчках осуществляется вызов существующей геометрии, после чего поочередно очищаются контейнеры с физическими, логическими и геометрическими объемами.

```
G4VPhysicalVolume* Geometry::Construct() {
    G4GeometryManager::GetInstance()->OpenGeometry();
    G4PhysicalVolumeStore::GetInstance()->Clean();
    G4LogicalVolumeStore::GetInstance()->Clean();
    G4SolidStore::GetInstance()->Clean();

    G4double size = 50 * m;
    world = new G4Box("world", size / 2., size / 2., size / 2.);
    world_log = new G4LogicalVolume(world, world_mat, "world_log");
    world_VP = new G4PVPlacement(0, G4ThreeVector(), world_log, "world_PV", 0, false, 0);

    G4Box *box = new G4Box("box", det_size / 2., det_size / 2., det_size / 2.);
    G4LogicalVolume *box_log = new G4LogicalVolume(box, box_mat, "box_log");
    new G4PVPlacement(0,G4ThreeVector(0,0,0), box_log, "pvp_box", world_log, false, 0);

    return world_VP;
}
```

Оптимизация навигации при изменении геометрии

Метод для установки нового размера (det_size) куба

```
void Geometry::SetDetSize(G4double newValue){
    det_size = newValue;
    G4RunManager::GetRunManager()->DefineWorldVolume(world_VP,true);
    G4RunManager::GetRunManager()->ReinitializeGeometry();
}
```

при изменении параметра необходимо сообщить **RunManager** о перестроении геометрии

Создание новых команд

Создание команды setSize в конструкторе класса GeometryMessenger

```
GeometryMessenger::GeometryMessenger(Geometry* Det):det(Det){
    my_dir = new G4UIdirectory("/my_dir/");
    my_dir ->SetGuidance("UI commands specific to this example");

    setSize = new G4UIcmdWithADouble("/my_dir/setCubeSize",this);
    setSize->SetGuidance("Select Size of the box.");
    setSize->SetParameterName("size",false);
}
```

Чтобы команда изменила размер куба, нужно реализовать в классе GeometryMessenger метод SetNewValue() и передавать в него значение из команды в случае её вызова.

```
void GeometryMessenger::SetNewValue(G4UIcommand *command, G4String newValue) {
    if (command == setSize) {
        det->SetDetSize(setSize->GetNewDoubleValue(newValue));
    }
}
```

В методе осуществляется проверка того, какая команда была вызвана, и при необходимости, передаваемое этой команде значение приводится к нужному формату.

преобразование к типу G4double осуществляет метод **G4cmdWithADouble::GetNewDoubleValue(G4String)**

Полученное значение передается объекту класса Geometry через ранее заданный метод **Geometry::SetDetSize(G4double)**, в результате чего запускается перезагрузка геометрии с новым значением

Создание новых команд

Для изменения материала куба, можно реализовать команду, которая принимает **G4String** значение:

```
setMaterial = new G4UIcmdWithAString("/my_dir/setMaterial",this);
setMaterial->SetGuidance("Select Material of the box.");
setMaterial->SetParameterName("name",false);
```

```
setMaterial = new G4UIcommand("/my_dir/setMaterial",this);
setMaterial->SetGuidance("Select Material of the box.");

G4UIparameter* mat = new G4UIparameter("name",'s',true);
mat->SetGuidance("Name from NIST");
mat->SetDefaultValue("G4_Li");
mat->SetParameterCandidates("G4_Na G4_Mn");

setMaterial->SetParameter(mat);
```

где **G4UIcommand**() представляет собой конструктор команды без параметра,

а **G4UIparameter*** является указателем на объект класса параметров команд

где в конструкторе **G4UIparameter**(**const char** * *theName*, **char** *theType*, **G4bool** *theOmittable*):

- *theName* – имя параметра
- *theType* – тип параметра задаваемый символом (*в данном случае это «строка»*)
- *theOmittable* – **true**/**false** в зависимости от того можно/нельзя вызывать команду без параметра

Создание новых команд

Для того чтобы передать новое значение в **SetNewValue(G4UIcommand*, G4String)**:

```
if (command == setMaterial){
    det->SetDetMaterial(newValue);
}
```

Где **Geometry::SetDetMaterial(G4String)** выглядит следующим образом:

```
57  void Geometry::SetDetMaterial(G4String newValue){
58      box_mat = nist->FindOrBuildMaterial(newValue);
59      G4RunManager::GetRunManager()->DefineWorldVolume(Construct());
60      G4RunManager::GetRunManager()->ReinitializeGeometry();
61  }
```

При выборе созданной команды из списка отобразится следующее сообщение в окне описания команд:

Command /my_dir/setMaterial
Guidance : Select Material of the box.

	Parameter	Guidance	Type	Ommitable	Default	Range	Candidate
1	name	Name from NIST	s	True	G4_Li		G4_Na G4_Mn